so for activating S_{act} and neutral S_{neu},

$$\frac{m_{\text{proj}}(S_{\text{act}})}{m_{\text{proj}}(S_{\text{neu}})} = \frac{\sum_{x \in S_{\text{act}}} \|\mathbf{x}\| \cos\theta_x}{\sum_{y \in S_{\text{neu}}} \|\mathbf{y}\| \cos\phi_y} \tag{8}$$

depends on the norms $\|\mathbf{x}\|$, $\|\mathbf{y}\|$. By contrast,

$$m_{\cos}(S) = \frac{1}{|S|} \sum_{\mathbf{x} \in S} \frac{\mathbf{f}^\top \mathbf{x}}{\|\mathbf{f}\| \|\mathbf{x}\|} = \frac{1}{|S|} \sum_{\mathbf{x} \in S} \cos\theta_{\mathbf{x}} \tag{9}$$

cancels $\|\mathbf{f}\|$ and $\|\mathbf{x}\|$. Since both unsupervised methods compare against different activating and neutral sets, using cosine similarity makes the activating/neutral ratio directly comparable across methods.

C SAE Training on MLP Activations

For causal evaluation we trained sparse autoencoders (SAEs) on MLP activations from LLaMA-3.1–8B at layers 0, 8, 16, 23, and 31 and Gemma-2–2B at layers 0, 6, 12, 18, and 25. For concept detection we additionally trained on layers 6, 12, 18, and 25 for LLaMA-3.1–8B. For training, we followed best practices as outlined in Gao et al. (2025). We trained both TopK and Gated architectures across a grid of hyperparameters: three learning rates $(1\text{e}{-4},\ 5\text{e}{-4},\ 1\text{e}{-5})$ and three sparsity levels per architecture. For TopK we used $k \in 3, 5, 10$; for Gated, we used L_1 penalties from $(5, 10, 20)$. Each SAE had 100 latents and was trained for 500 epochs. We observed that low regularization tended to produce overly dense features, while increased regularization led to large amounts of dead features. These challenges were particularly difficult to balance with the limited settings, where the smaller number of features (100) and smaller dataset made training less stable and led to training that was more sensitive to hyperparameter choice. To address this, we manually identified a reasonable lower and upper bound for the hyperparameters that didn't create an obvious collapse of training.

We saved 10 checkpoints per run and evaluated them on a held-out validation set constructed identically to the training set, with four sentences per concept. To select the best checkpoint, we combined two metrics: (1) the average number of nonzero activations (to capture sparsity) and (2) downstream loss, measured by the KL divergence between the original model logits and the logits after injecting SAE reconstructions (Gao et al., 2025). We normalized both metrics and combined them to form the selection objective. For Gemma, giving weight to sparsity led to a large amount of dead features, so only downstream loss was used for checkpoint selection. Overall, we trained 252 SAEs and evaluated 2520 checkpoints to identify the SAEs used.

D Comparison of Interpretability Results Across Four K Values

Here we describe the experimental settings of the concept detection benchmarking (§4) and provide comparisons for varying values of k.

D.1 SNMF Dataset

To construct the SNMF training data we randomly sampled 20 concepts from Neuronpedia (Lin, 2023). For each concepts we prompted GPT-4o-mini (OpenAI et al., 2024) to generate 10 unique sentences exemplifying the concept. Overall, the resulting dataset contains 200 sentences exemplifying 20 labeled concepts and additional non-labeled concepts. For the prompt used to generate the dataset refer to §G.1.

D.2 SNMF Training

We applied SNMF as described in §A with varying values of $k = 100, 200, 300, 400$. For Llama-3.1-8B and Gemma-2-2B, we kept the top 1 percent of indices in each MLP feature during training as described in §A, whereas for GPT-2 we keep the top 5 percent due to its smaller dimensions.

D.3 SAE Concepts

For the SAEs, we randomly sampled 100 features per layer from those identified by Neuronpedia (Lin, 2023), using the SAEs from Llamascope, Gemmascope, and OpenAI's GPT-2 Small (He et al., 2024; Lieberum et al., 2024; Gao et al., 2025), all trained on MLP-out.

D.4 Evaluation

The evaluation procedure is identical across all methods and layers. For each concept and layer, we follow the same methodology: generate 5 activating sentences that exemplify the concept and 5 neutral sentences that do not. Each sentence is passed through the model, and we extract the activations at the target layer. For each activation, we compute the cosine similarity with the feature and record the maximum similarity across all tokens in the sentence. We then average these maximum scores across the activating sentences to obtain $\bar{a}$activating, and across the neutral sentences to

obtain $\bar{a}$neutral. Finally, we compute the Concept Detection score (S_{CD}) as the log-ratio between the two averages:

$$S_{CD} := \log \frac{\bar{a}\text{activating}}{\bar{a}\text{neutral}} \tag{10}$$

D.5 Results

Figures 6, 7 and 8, present results for varying values of k in SNMF, while for SAE the number of concepts is fixed at 100. Across all models, we observe that increasing k generally improves cosine similarity in most layers, suggesting that larger SNMF decompositions yield features that more cleanly activate for specific concepts. However, there are exceptions. For instance, in Gemma-2-2B at layer 12, performance degrades as k increases. This may indicate that setting k higher than the number of meaningful concepts present at a given layer can hurt performance.

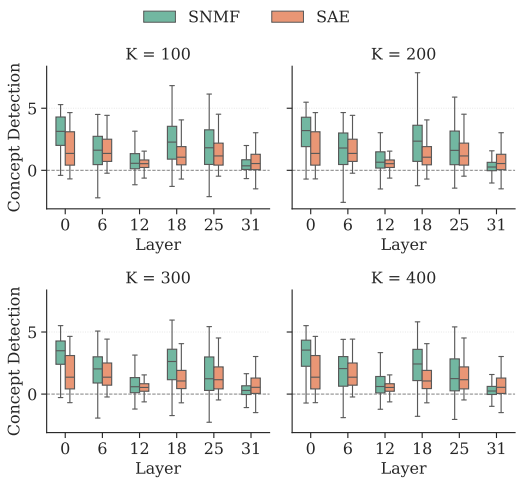

Figure 6: Interpretability scores for Llama-3.1-8B across $K = 100, 200, 300, 400$.

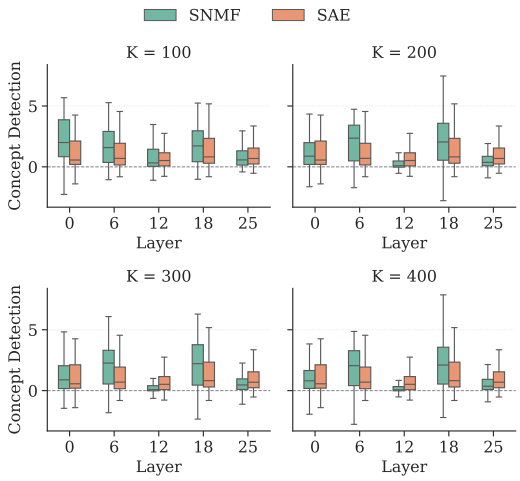

Figure 7: Interpretability scores for Gemma-2-2B across $K = 100, 200, 300, 400$.

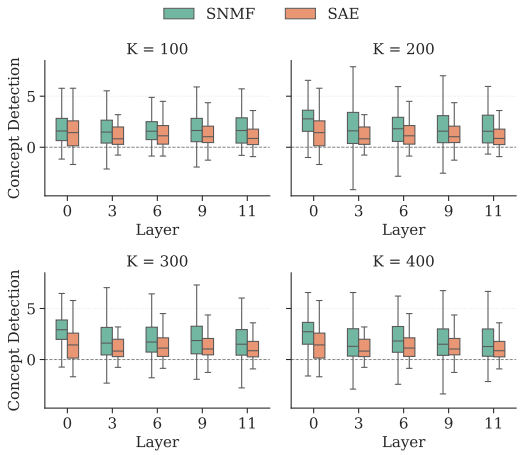

Figure 8: Interpretability scores for GPT-2 Small across $K = 100, 200, 300, 400$.

E Fluency Scores of Causal Benchmark

Figure 9 presents the Fluency score results from §5. Notably, we see a low score for Layer 0 showing the increased difficulty in maintaining model coherence at lower layer interventions.

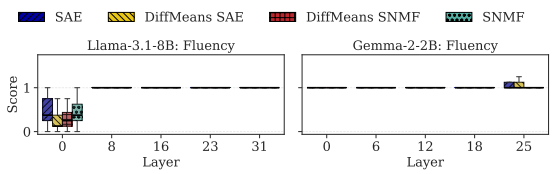

Figure 9: Fluency scores for Llama-3.1-8B and Gemma-2-2B across all methods: SAE, DiffMeans trained on SAE concepts, DiffMeans trained on SNMF concepts and SNMF.

F Experimental Details of Neuron Compositionality

F.1 Recursive SNMF Algorithm

We follow the method outlined in Song and Lee (2013). Initially, we decompose the activations as described in §A to obtain matrices Y_0 and Z_0. Then we recursively decompose the matrix Z_i to produce matrices Y_{i+1} and Z_{i+1}. To improve the integration of the different MLP feature hierarchies, following Song and Lee (2013), we jointly fine-tune all layers using gradient descent to minimize the reconstruction loss between the original activations A and their multi-layer approximation:

$$\mathcal{L} = \frac{1}{2} \|A - Z_L Y_L \cdots Y_1 Y_0\|_F^2$$

The gradients are propagated backward through the factorization layers, and each Z_i and Y_i is updated using gradient descent. This yields a set of multi-level MLP features, where higher-layer features encode increasingly abstract combinations of co-activated neurons.

F.2 Dataset Generation: Time Units

We prompted GPT-o4-mini-high to generate a dataset focused on the concept of units of time: years, months, weekdays, minutes etc. We chose this concept because prior work (Engels et al., 2025) has shown that GPT-2 represents weekdays and months in a multi-dimensional manner, where each day of the week is positioned near the ones that follow it, forming a circular structure in a lower-dimensional subspace. We use the following prompt:

Dataset Generation Prompt

Generate a dataset of sentences that are all related to days of the week, months, years, dates etc. Make sure to include plenty of both broad and specific terms including all names of the days and months and for each term over 10 instances. Generate 200 sentences. Format the dataset with two columns: Prompt, Label. Format as a json in the format <label>: [<sentences>].

F.3 Dataset Generation: General

We constructed a more general dataset by prompting GPT-4o for 50 random nouns as base concepts. For each noun, we queried ConceptNet to retrieve related concepts that represent subcomponents or subcategories using relations like HasA, PartOf, or TypeOf. For each of the related concepts, we prompted Gemini-2.0-flash (Google, 2025) to generate five distinct sentences exemplifying the relationship in natural language. This process resulted in a total of 873 sentences that reflect a diverse range of semantically related sentence groups.

F.4 Tree Construction

To identify hierarchical MLP features, we apply recursive SNMF with $K = [400, 200, 100, 50]$. Each matrix Y_i maps features from level $i{+}1$ to those in Z_i, allowing us to trace how higher-level features activate lower-level ones. To construct the tree, we connect each feature at level $i{+}1$ to its top-activated features in Z_i according to Y_i, using a fixed threshold to prune weak edges. This forms a directed tree structure that reflects the compositional organization of features across levels. We label the nodes corresponding to the MLP features using the top activating contexts associated with the MLP features. We obtain the mapping for an MLP feature at layer i in the hierarchy to the contexts by multiplying the coefficient matrices 0 to i: $Y^{(0\to i)} = Y_0 Y_1 \cdots Y_i$

F.5 Additional Hierarchical Decompositions

In Figures 10, 11, 12, 13 and 14 we provide additional examples of hierarchical MLP features found by applying the described methodology on the general dataset on MLP activations extracted from Layer 0. We manually select the examples to provide a variety of compositions. We note that these are a select few from a larger pool of identified hierarchical MLP features.

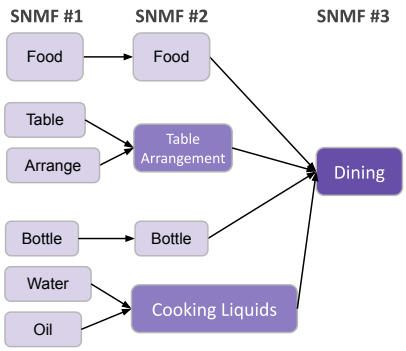

Figure 10: Merging of Dining related MLP features.